

E-COMMERCE LABORATORY

Complete Lab Report: Programs 1 to 6

Labs 1 to 5 are Python programs. Lab 6 studies Google Analytics 4 on Google's live Merchandise Store demo.

Name	Abhishek Barali
Roll No.	023bscit003
Section	A
Semester / Batch	2023
Subject	E-Commerce Laboratory
Language used	Python 3 with the Flask micro framework
Data storage	Lists and dictionaries inside each program, no database

Source code: <https://github.com/AbhishekBarali/ecommerce-lab>

Contents

1. Introduction
2. How the programs are organised and run
3. Lab 1: Product category list and items
4. Lab 2: Product list and add to cart
5. Lab 3: Add to cart and checkout
6. Lab 4: Product list and wish list
7. Lab 5: Payment gateway integration in sandbox mode
8. Lab 6: Google Analytics setup and analytics parameters
9. Conclusion

Introduction

This report covers all six labs of the E-Commerce Laboratory. Labs 1 to 5 are separate programs, each with its own folder, product data and interface, written in Python with the Flask micro framework and keeping their records in ordinary Python lists and dictionaries (no database, as the assignment allows). Lab 6 is different: it studies Google Analytics 4 directly on Google's public Merchandise Store demo, with no program to write.

Each lab is presented with its objective, the logic that was used, the full source code of the program, and screenshots taken while the program was running. The store data carries the name Abhishek Barali and the roll number 023bscit003 so the output belongs to this submission.

How the programs are organised and run

Each folder holds one program: app.py with the logic, and a templates folder with the page it renders. Every lab listens on its own port, so several labs can run side by side.

Folder	Program	Port
lab1_categories	Categories and items	5001
lab2_cart	Product list and cart	5002
lab3_checkout	Cart and checkout	5003
lab4_wishlist	Wish list	5004
lab5_payment	Payment gateway, sandbox	5005
Lab 6	Google Analytics 4, studied on Google's live demo (no program)	-

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

```
cd lab1_categories    # or lab2_cart, lab3_checkout, lab4_wishlist,
.venv/bin/python app.py # lab5_payment, lab6_analytics
```

Then open the port printed by the program, for example <http://127.0.0.1:5001>

Commands used to install Flask and run one lab

Lab 1: Product Category List and Items

Program folder: `lab1_categories` | run with `"python app.py"` and open <http://127.0.0.1:5001>

Objective

Create a product category list, add items into the created categories, and display all items together with their categories. The program is presented as the catalogue desk of a shop called Barali Traders.

Logic used

1. Categories are held in a plain list of names, and items in a list of dictionaries where every item stores the category it belongs to.
2. The category form appends a new name to the category list, and a name that already exists is reported instead of being added twice.
3. The item form appends a dictionary with a code, name, price, stock and the category chosen from the dropdown.
4. The item code is generated from the first two letters of the category and the item count, for example ST-08 for the eighth item filed under Stationery.
5. For display, the items are grouped category by category and each group is shown as its own table, so every item appears next to its category.

Source code

```
"""Lab 1 - Product category list and items.
```

```
Create product categories, add items into a chosen category, and display
every item together with its category.
```

```
Run: python app.py    then open http://127.0.0.1:5001
```

```
Abhishek Barali - 023bscit003 - Section A - Batch 2023
```

```
"""
```

```
from flask import Flask, render_template, request, redirect, url_for
```

```
app = Flask(__name__)
```

```

STAFF = {"name": "Abhishek Barali", "roll": "023bscit003"}

# Categories are a plain list of names.
categories = ["Electronics", "Fashion", "Groceries"]

# Items are a list of dictionaries; each one remembers its category.
items = [
    {"code": "BT-01", "name": "Bluetooth Headphone", "price": 2500,
     "stock": 12, "category": "Electronics"},
    {"code": "PB-02", "name": "Power Bank 10000mAh", "price": 1800,
     "stock": 7, "category": "Electronics"},
    {"code": "SW-03", "name": "Smart Watch", "price": 4200,
     "stock": 4, "category": "Electronics"},
    {"code": "TS-04", "name": "Cotton T-Shirt", "price": 900,
     "stock": 25, "category": "Fashion"},
    {"code": "RS-05", "name": "Running Shoes", "price": 3500,
     "stock": 9, "category": "Fashion"},
    {"code": "BR-06", "name": "Basmati Rice 5kg", "price": 1250,
     "stock": 30, "category": "Groceries"},
    {"code": "SO-07", "name": "Sunflower Oil 2L", "price": 640,
     "stock": 18, "category": "Groceries"},
]

def next_code(category):
    """Build a short item code from the category name and the item count."""
    return f'{category[:2].upper()}-{len(items) + 1:02d}'

@app.route("/")
def home():
    grouped = [(c, [i for i in items if i["category"] == c]) for c in categories]
    return render_template("index.html", staff=STAFF, categories=categories,
                          grouped=grouped, items=items,
                          note=request.args.get("note"))

@app.route("/category", methods=["POST"])
def add_category():
    name = request.form.get("category", "").strip()
    if not name:
        return redirect(url_for("home"))
    if name in categories:
        return redirect(url_for("home", note=f'Category '{name}' already exists.'))
    categories.append(name)
    return redirect(url_for("home", note=f'Category '{name}' created.'))

@app.route("/item", methods=["POST"])
def add_item():
    name = request.form.get("name", "").strip()
    category = request.form.get("category", "")
    if not name or category not in categories:
        return redirect(url_for("home"))
    items.append({
        "code": next_code(category),
        "name": name,
        "price": int(request.form.get("price") or 0),
    })

```

```

"stock": int(request.form.get("stock") or 0),
"category": category,
})
return redirect(url_for("home", note=f"'{name}' added under {category}."))

```

```

if __name__ == "__main__":
    app.run(port=5001, debug=True)

```

lab1_categories/app.py

Output

The category 'Stationery' was created, the item 'A4 Notebook' was filed under it, and the catalogue then listed every item under its own category with the item count per category.

BARALI TRADERS
Catalogue Desk — Lab 1: product categories and items
Maintained by Abhishek Barali | Roll No. 023bsci003 | Section A, Batch 2023

CREATE AND FILL A CATEGORY

NEW CATEGORY

Category name

Create category

NEW ITEM

Item name

Price in NPR

Stock

Category

Electronics

Add item

ITEMS WITH THEIR CATEGORIES — 7 IN TOTAL

Electronics 3 item(s)				
CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
BT-01	Bluetooth Headphone	Electronics	12	2500
PB-02	Power Bank 10000mAh	Electronics	7	1800
SW-03	Smart Watch	Electronics	4	4200

Fashion 2 item(s)				
CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
TS-04	Cotton T-Shirt	Fashion	25	900
RS-05	Running Shoes	Fashion	9	3500

Groceries 2 item(s)				
CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
BR-06	Basmati Rice 5kg	Groceries	30	1250
SO-07	Sunflower Oil 2L	Groceries	18	640

Lab 1 program. Categories and items are held in a Python list and a list of dictionaries, no database.

Figure 1: Lab 1 before any change: three categories and seven items.

BARALI TRADERS

Catalogue Desk — Lab 1: product categories and items

Maintained by Abhishek Barali | Roll No. 023bsci003 | Section A, Batch 2023

'A4 Notebook' added under Stationery.

CREATE AND FILL A CATEGORY

NEW CATEGORY

Category name

Stationery

Create category

NEW ITEM

Item name

A4 Notebook

Price in NPR

220

Stock

40

Category

Electronics

Add item

ITEMS WITH THEIR CATEGORIES — 8 IN TOTAL

Electronics3 item(s)

CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
BT-01	Bluetooth Headphone	Electronics	12	2500
PB-02	Power Bank 10000mAh	Electronics	7	1800
SW-03	Smart Watch	Electronics	4	4200

Fashion2 item(s)

CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
TS-04	Cotton T-Shirt	Fashion	25	900
RS-05	Running Shoes	Fashion	9	3500

Groceries2 item(s)

CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
BR-06	Basmati Rice 5kg	Groceries	30	1250
SO-07	Sunflower Oil 2L	Groceries	18	640

Stationery1 item(s)

CODE	ITEM	CATEGORY	STOCK	PRICE (NPR)
ST-08	A4 Notebook	Stationery	40	220

Lab 1 program. Categories and items are held in a Python list and a list of dictionaries, no database.

Figure 2: Lab 1 output. 'Stationery' was created and 'A4 Notebook' was added to it, so the catalogue now shows four categories and eight items.

Lab 2: Product List and Add to Cart

Program folder: lab2_cart | run with "python app.py" and open <http://127.0.0.1:5002>

Objective

Create a product list, provide an add-to-cart feature for the products, and display the cart items. The program is presented as an online grocery called Hariyo Bazaar, with the cart kept beside the shelves.

Logic used

1. Products are a list of dictionaries and are drawn shelf by shelf, so the same list produces the Grains, Beverages and Dairy sections.
2. The cart is a separate list; each cart line stores the product id, name, unit, price and quantity.
3. Adding a product that is already in the cart raises its quantity instead of creating a second line.
4. The minus button lowers the quantity, and a line that reaches zero is dropped from the cart.
5. The cart panel shows every line with its own amount, and the total is the sum of price multiplied by quantity across all lines.

Source code

```
"""Lab 2 - Product list and add to cart.

Show a product list, add products to the cart, change quantity, and display
the cart items with a total.

Run: python app.py   then open http://127.0.0.1:5002

Abhishek Barali - 023bscit003 - Section A - Batch 2023
"""
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

SHOPPER = {"name": "Abhishek Barali", "roll": "023bscit003", "city": "Kathmandu"}

products = [
    {"id": 1, "name": "Basmati Rice", "unit": "5 kg bag", "price": 1250, "shelf":
    "Grains"},
    {"id": 2, "name": "Sunflower Oil", "unit": "2 litre", "price": 640, "shelf":
    "Grains"},
    {"id": 3, "name": "Green Tea", "unit": "100 g", "price": 320, "shelf": "Beverages"},
    {"id": 4, "name": "Ground Coffee", "unit": "250 g", "price": 780, "shelf":
    "Beverages"},
    {"id": 5, "name": "Cheddar Cheese", "unit": "400 g", "price": 910, "shelf":
    "Dairy"},
    {"id": 6, "name": "Fresh Curd", "unit": "500 ml", "price": 180, "shelf": "Dairy"},
]

cart = []   # each line: {"id", "name", "unit", "price", "qty"}

def find(product_id):
```

```

    return next((p for p in products if p["id"] == product_id), None)

def line_for(product_id):
    return next((line for line in cart if line["id"] == product_id), None)

def total():
    return sum(line["price"] * line["qty"] for line in cart)

@app.route("/")
def home():
    shelves = sorted({p["shelf"] for p in products}, key=lambda s: s)
    return render_template("index.html", shopper=SHOPPER, products=products,
                           shelves=shelves, cart=cart, total=total(),
                           items=sum(line["qty"] for line in cart),
                           note=request.args.get("note"))

@app.route("/add/<int:product_id>", methods=["POST"])
def add(product_id):
    product = find(product_id)
    if not product:
        return redirect(url_for("home"))
    line = line_for(product_id)
    if line:
        line["qty"] += 1          # already in the cart, so raise qty
    else:
        cart.append({"id": product["id"], "name": product["name"],
                     "unit": product["unit"], "price": product["price"], "qty": 1})
    return redirect(url_for("home", note=f'{product["name"]} added to the cart.))

@app.route("/less/<int:product_id>", methods=["POST"])
def less(product_id):
    line = line_for(product_id)
    if line:
        line["qty"] -= 1
        if line["qty"] <= 0:
            cart.remove(line)
    return redirect(url_for("home"))

@app.route("/drop/<int:product_id>", methods=["POST"])
def drop(product_id):
    line = line_for(product_id)
    if line:
        cart.remove(line)
        return redirect(url_for("home", note=f'{line["name"]} removed from the cart.))
    return redirect(url_for("home"))

if __name__ == "__main__":
    app.run(port=5002, debug=True)

```

lab2_cart/app.py

Output

Basmati Rice, Green Tea and Cheddar Cheese were added, and Green Tea was added twice so its line shows a quantity of two. The cart reported four items and a total of NPR 2800.

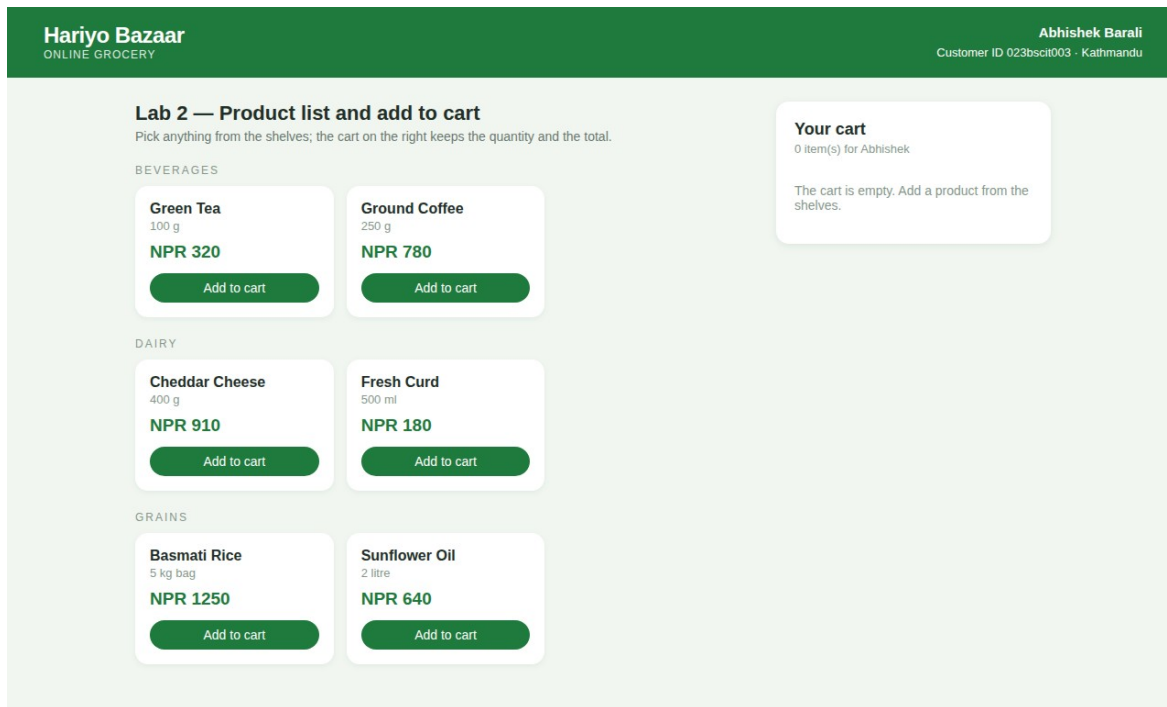


Figure 3: Lab 2 with an empty cart, showing the product shelves.

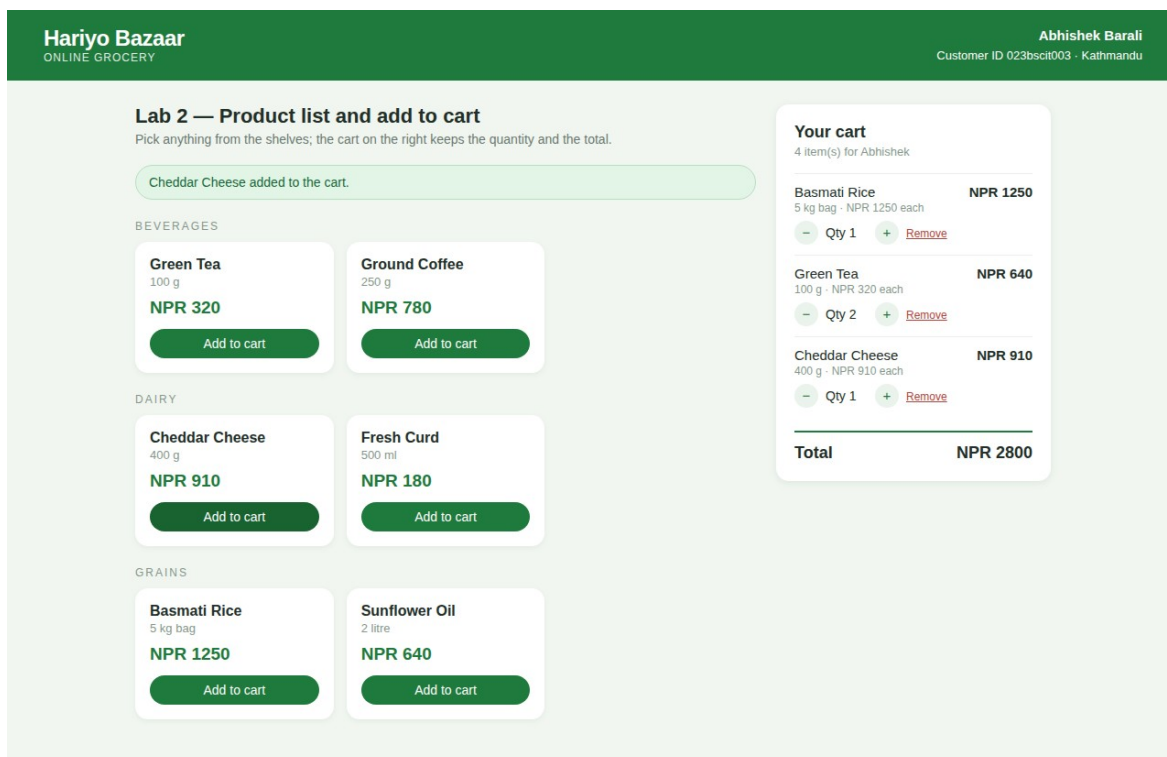


Figure 4: Lab 2 output. Three products in the cart, one of them with quantity two, and the total for the customer 023bscit003.

Lab 3: Add to Cart and Checkout

Program folder: `lab3_checkout` | run with `"python app.py"` and open `http://127.0.0.1:5003`

Objective

Implement the add-to-cart and checkout features of a shopping store and display the ordered list of items. The program is presented as the desk accessory store DeskWorks, and every completed order prints as a receipt.

Logic used

1. Products are added to a cart list in the same way as Lab 2, with repeat additions raising the quantity.
2. The payable amount is the goods total plus a fixed delivery charge of NPR 100.
3. The checkout form collects the customer name, delivery address and phone number, pre-filled with the details of Abhishek Barali.
4. On checkout an order dictionary is built with an order number, the customer details, the time, a copy of the cart lines and the totals.
5. The order number is generated from the roll number, so the first order is ORD-023BSCIT003-01.
6. The order is appended to the orders list and the cart is cleared, which is how a store behaves once an order is placed.
7. Every placed order is then printed below as an ordered list of items with rate, quantity and amount.

Source code

```
"""Lab 3 - Add to cart and checkout.

Add products to the cart, check out with customer details, and display the
ordered list of items as a receipt.

Run: python app.py   then open http://127.0.0.1:5003

Abhishek Barali - 023bscit003 - Section A - Batch 2023
"""
from datetime import datetime

from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)
```

```

BUYER = {"name": "Abhishek Barali", "roll": "023bscit003",
        "address": "Baneshwor, Kathmandu", "phone": "98XXXXXX01"}

DELIVERY = 100

products = [
    {"id": 1, "name": "Mechanical Keyboard", "price": 5400},
    {"id": 2, "name": "Wireless Mouse", "price": 1350},
    {"id": 3, "name": "Laptop Stand", "price": 2100},
    {"id": 4, "name": "USB-C Hub", "price": 2950},
    {"id": 5, "name": "Desk Lamp", "price": 1780},
    {"id": 6, "name": "Monitor Riser", "price": 3200},
]

cart = [] # each line: {"id", "name", "price", "qty"}
orders = [] # each order: {"number", "buyer", "roll", "address", "placed",
                        # "lines", "goods", "delivery", "total"}

def find(product_id):
    return next((p for p in products if p["id"] == product_id), None)

def goods_total():
    return sum(line["price"] * line["qty"] for line in cart)

@app.route("/")
def home():
    return render_template("index.html", buyer=BUYER, products=products,
                           cart=cart, goods=goods_total(), delivery=DELIVERY,
                           orders=list(reversed(orders)),
                           note=request.args.get("note"))

@app.route("/add/<int:product_id>", methods=["POST"])
def add(product_id):
    product = find(product_id)
    if not product:
        return redirect(url_for("home"))
    line = next((l for l in cart if l["id"] == product_id), None)
    if line:
        line["qty"] += 1
    else:
        cart.append({"id": product["id"], "name": product["name"],
                    "price": product["price"], "qty": 1})
    return redirect(url_for("home", note=f"{product['name']} is in the cart.))

@app.route("/drop/<int:product_id>", methods=["POST"])
def drop(product_id):
    cart[:] = [line for line in cart if line["id"] != product_id]
    return redirect(url_for("home", note="Item removed from the cart.))

@app.route("/checkout", methods=["POST"])
def checkout():

```

```

"""Turn the cart into an order, then empty the cart."""
if not cart:
    return redirect(url_for("home", note="The cart is empty, so there is nothing to
        check out.))

goods = goods_total()
order = {
    "number": f'ORD-{BUYER["roll"].upper()}-{len(orders) + 1:02d}',
    "buyer": request.form.get("name", BUYER["name"]).strip() or BUYER["name"],
    "roll": BUYER["roll"],
    "address": request.form.get("address", BUYER["address"]).strip() or
        BUYER["address"],
    "phone": request.form.get("phone", BUYER["phone"]).strip() or BUYER["phone"],
    "placed": datetime.now().strftime("%d %b %Y, %I:%M %p"),
    "lines": [dict(line) for line in cart],    # copy of the cart
    "goods": goods,
    "delivery": DELIVERY,
    "total": goods + DELIVERY,
}
orders.append(order)
cart.clear()
return redirect(url_for("home", note=f"Order {order['number']} placed.))

if __name__ == "__main__":
    app.run(port=5003, debug=True)

```

lab3_checkout/app.py

Output

One mechanical keyboard and two USB-C hubs were checked out as order ORD-023BSCIT003-01 for NPR 11,400 including delivery. The cart is empty afterwards and the order appears as a receipt.

DESKWORKS

Signed in as **Abhishek Barali**
Customer 023bscit003 - Section A, Batch 2023

Lab 3 — add to cart and checkout. The cart becomes an order, the cart empties, and the ordered items print below as a receipt.

USB-C Hub is in the cart.

PRODUCTS

Mechanical Keyboard
NPR 5400

Add to cart

Wireless Mouse
NPR 1350

Add to cart

Laptop Stand
NPR 2100

Add to cart

USB-C Hub
NPR 2950

Add to cart

Desk Lamp
NPR 1700

Add to cart

Monitor Riser
NPR 3200

Add to cart

CART

ITEM	QTY	AMOUNT	
Mechanical Keyboard	1	5400	remove
USB-C Hub	2	5900	remove
Delivery		100	
Payable		11400	

CHECKOUT

CUSTOMER NAME

Abhishek Barali

DELIVERY ADDRESS

Baneshwor, Kathmandu

PHONE

98XXXXXX01

Place order

ORDERED LIST OF ITEMS

No order has been placed yet.

Figure 5: Lab 3 before checkout: the cart holds three units and shows the payable amount.

DESKWORKS

Signed in as **Abhishek Barali**
Customer 023bscit003 - Section A, Batch 2023

Lab 3 — add to cart and checkout. The cart becomes an order, the cart empties, and the ordered items print below as a receipt.

Order ORD-023BSCIT003-01 placed.

PRODUCTS

Mechanical Keyboard
NPR 5400
Add to cart

Wireless Mouse
NPR 1350
Add to cart

Laptop Stand
NPR 2100
Add to cart

USB-C Hub
NPR 2950
Add to cart

Desk Lamp
NPR 1700
Add to cart

Monitor Riser
NPR 3200
Add to cart

CART

Cart is empty.

CHECKOUT

CUSTOMER NAME
Abhishek Barali

DELIVERY ADDRESS
Baneshwor, Kathmandu

PHONE
98XXXXXX01

Place order

ORDERED LIST OF ITEMS

ORD-023BSCIT003-01

03 Sep 2026, 02:56 AM
Customer 023bscit003

Abhishek Barali
Baneshwor, Kathmandu
98XXXXXX01

ITEM	RATE	QTY	AMOUNT
Mechanical Keyboard	5400	1	5400
USB-C Hub	2950	2	5900
Goods			11300
Delivery			100
TOTAL PAID			NPR 11400

Figure 6: Lab 3 output. Order ORD-023BSCIT003-01 was placed, the cart is empty, and the ordered list of items is printed as a receipt.

Lab 4: Product List and Wish List

Program folder: lab4_wishlist | run with "python app.py" and open <http://127.0.0.1:5004>

Objective

Create a product list with an add-to-wish-list feature and display the wish list items. The program is presented as WishBox, a saved-items page belonging to Abhishek Barali.

Logic used

1. The wish list is a list separate from the cart, so a product can be kept for later without being bought.
2. Saving a product copies its dictionary and stamps the time it was saved, which is shown next to the item.
3. A product already on the wish list cannot be saved twice; its button changes to 'Already on wish list'.
4. A saved item can be removed, or moved to the cart, in which case it is appended to the cart list and dropped from the wish list.
5. The wish list shows the saved items in order with their brand, saved time and price, and adds up the value being held.

Source code

```
"""Lab 4 - Product list and wish list.

Save products to a wish list, display the saved items, remove them, and move
a saved item into the cart.

Run: python app.py   then open http://127.0.0.1:5004

Abhishek Barali - 023bscit003 - Section A - Batch 2023
"""
from datetime import datetime

from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

OWNER = {"name": "Abhishek Barali", "roll": "023bscit003"}

products = [
    {"id": 1, "name": "Acoustic Guitar", "brand": "Cort", "price": 24500},
    {"id": 2, "name": "Noise Cancelling Headphone", "brand": "Sony", "price": 18900},
    {"id": 3, "name": "Fitness Band", "brand": "Mi", "price": 4300},
    {"id": 4, "name": "Espresso Maker", "brand": "Delonghi", "price": 15600},
    {"id": 5, "name": "Trekking Backpack 45L", "brand": "Quechua", "price": 7200},
    {"id": 6, "name": "Instant Camera", "brand": "Fujifilm", "price": 11800},
]

wishlist = [] # each entry: product dictionary plus the time it was saved
cart = []    # products moved out of the wish list

def find(product_id):
    return next((p for p in products if p["id"] == product_id), None)

def saved_ids():
    return [w["id"] for w in wishlist]
```

```

@app.route("/")
def home():
    return render_template("index.html", owner=OWNER, products=products,
                           wishlist=wishlist, saved=saved_ids(), cart=cart,
                           value=sum(w["price"] for w in wishlist),
                           note=request.args.get("note"))

@app.route("/save/<int:product_id>", methods=["POST"])
def save(product_id):
    product = find(product_id)
    if not product:
        return redirect(url_for("home"))
    if product["id"] in saved_ids():
        return redirect(url_for("home", note=f'{product["name"]} is already on the wish
            list.'))
    entry = dict(product)
    entry["saved_at"] = datetime.now().strftime("%d %b, %I:%M %p")
    wishlist.append(entry)
    return redirect(url_for("home", note=f'{product["name"]} saved to the wish list.'))

@app.route("/forget/<int:product_id>", methods=["POST"])
def forget(product_id):
    wishlist[:] = [w for w in wishlist if w["id"] != product_id]
    return redirect(url_for("home", note="Item removed from the wish list.'))

@app.route("/move/<int:product_id>", methods=["POST"])
def move(product_id):
    """Move a saved item out of the wish list and into the cart."""
    entry = next((w for w in wishlist if w["id"] == product_id), None)
    if entry:
        cart.append(entry)
        wishlist.remove(entry)
        return redirect(url_for("home", note=f'{entry["name"]} moved to the cart.'))
    return redirect(url_for("home"))

if __name__ == "__main__":
    app.run(port=5004, debug=True)

```

lab4_wishlist/app.py

Output

Three products were saved, holding a value of NPR 51,900, and their buttons changed to 'Already on wish list'. The espresso maker was then moved to the cart, which left two items on the wish list.

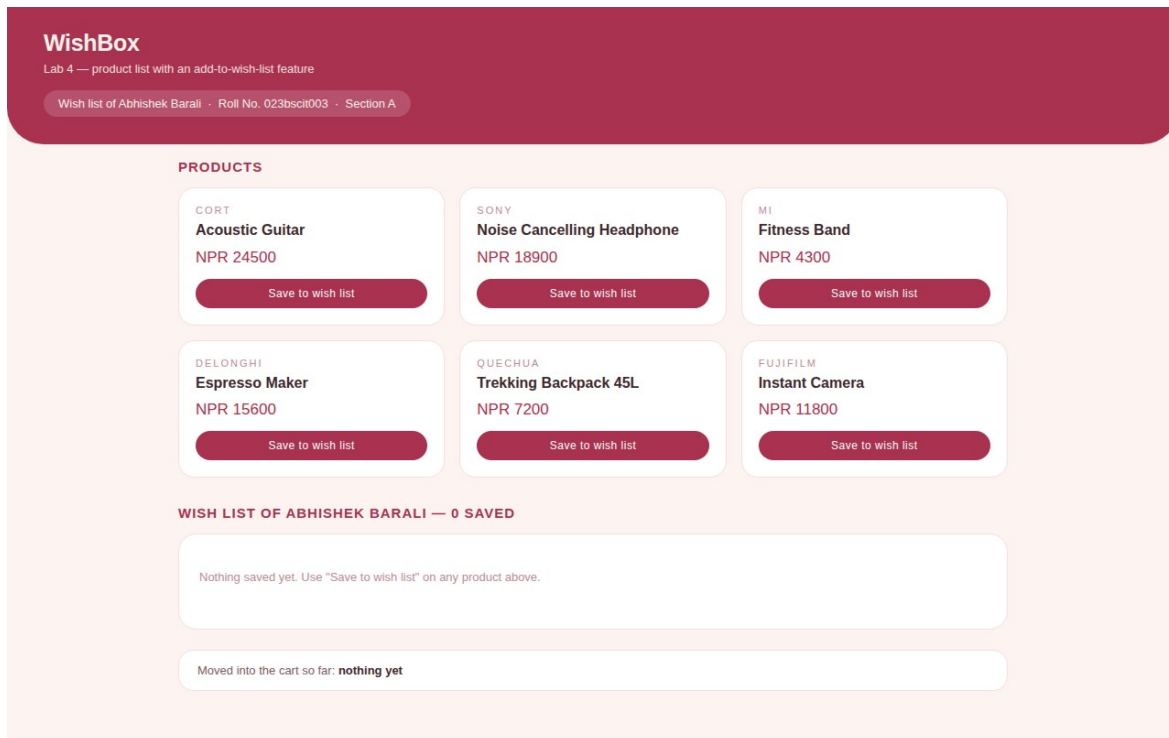


Figure 7: Lab 4 with an empty wish list.

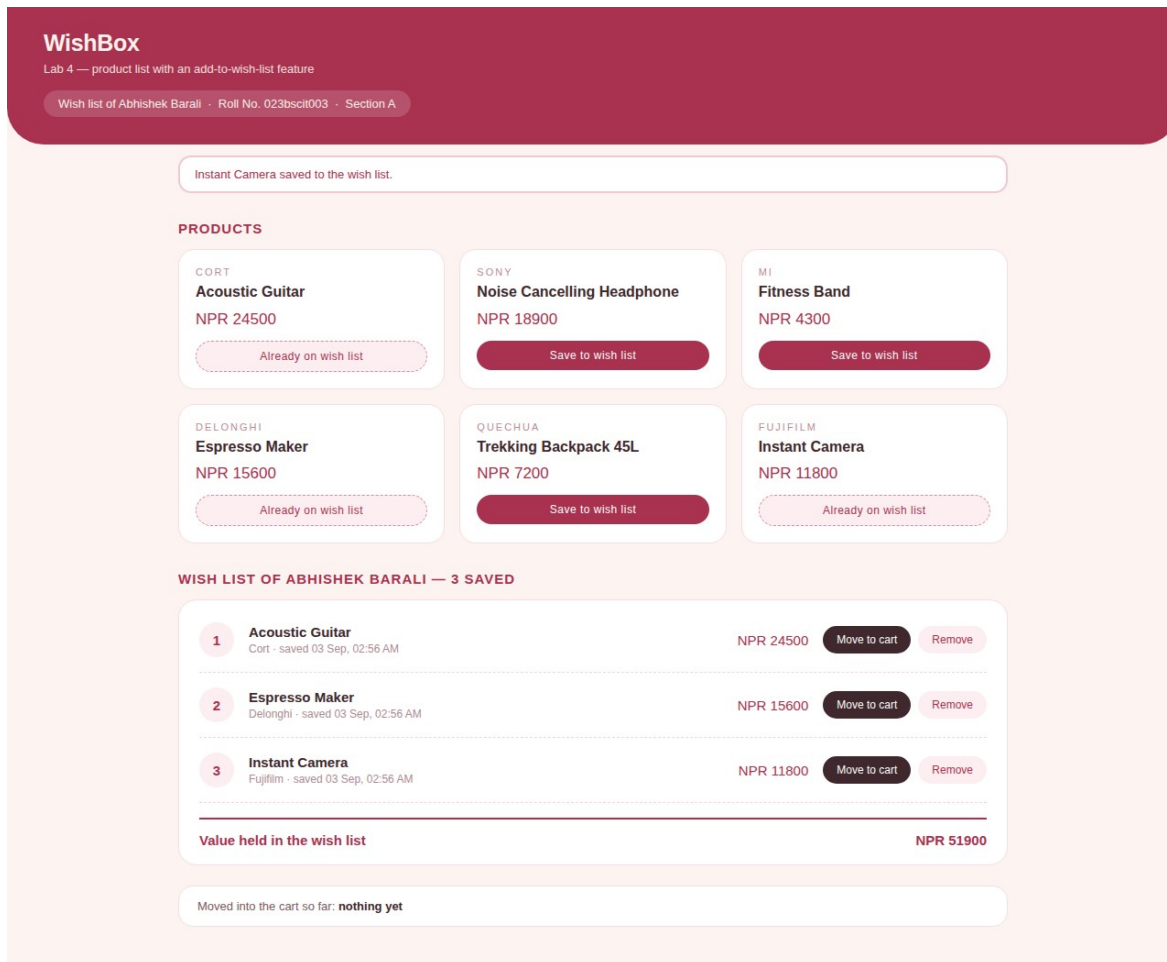


Figure 8: Lab 4 output. Three items saved to the wish list of Abhishek Barali with the total value held.

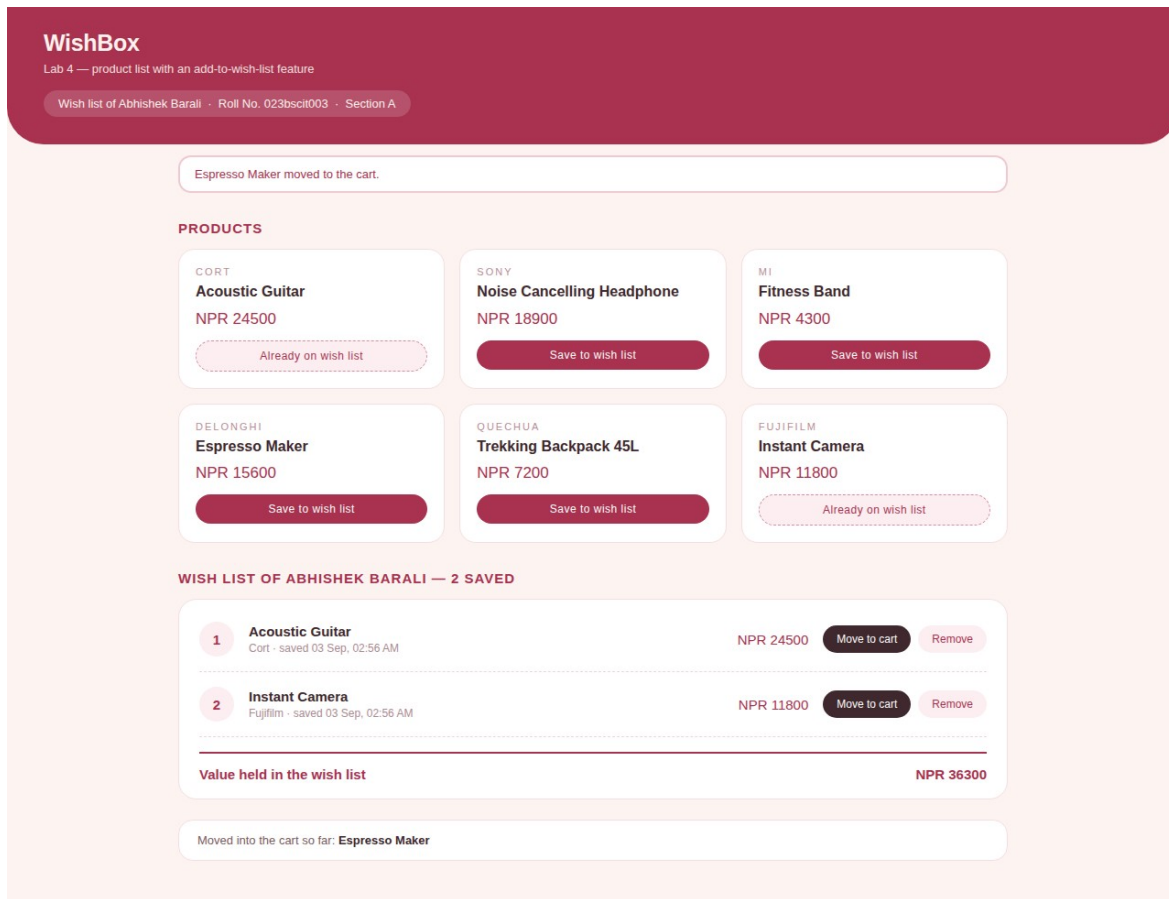


Figure 9: Lab 4 after 'Move to cart' on the espresso maker: the wish list drops to two items and the cart strip names the moved product.

Lab 5: Payment Gateway Integration (Sandbox)

Program folder: `lab5_payment` | run with `"python app.py"` and open `http://127.0.0.1:5005`

Objective

Integrate a payment gateway of choice into an e-commerce page. eSewa was chosen and is used in sandbox (test) mode, so the whole payment flow can be shown without a live merchant key and without any real money.

Logic used

1. The invoice page adds up the invoice lines and shows the payable amount for the customer 023bscit003.
2. Pressing the pay button posts the amount and the merchant product code EPAYTEST to the gateway, which is how the live integration also begins.
3. The gateway screen asks for the eSewa ID, password and token. The published sandbox test values are pre-filled.

4. The program verifies the three values and builds a payment record with a transaction reference, the amount, the channel, the time and a status.
5. The status is COMPLETE when the credentials match and FAILED when they do not, so both paths of a real gateway can be demonstrated.
6. The result page shows the response from the gateway, and every attempt is kept in a payment log.

Source code

""Lab 5 - Payment gateway integration in sandbox (test) mode.

The store hands the amount to the gateway, the customer confirms with the published eSewa sandbox test credentials, and the gateway returns a transaction reference with a status. No live merchant key is used and no real money moves, so the flow can be demonstrated offline.

Run: python app.py then open http://127.0.0.1:5005

Abhishek Barali - 023bscit003 - Section A - Batch 2023

"""

```
import random
import string
from datetime import datetime
```

```
from flask import Flask, render_template, request, redirect, url_for
```

```
app = Flask(__name__)
```

```
CUSTOMER = {"name": "Abhishek Barali", "roll": "023bscit003",
            "email": "abhishek.barali@example.com"}
```

```
# Published eSewa sandbox values. Test environment only.
```

```
SANDBOX = {"product_code": "EPAYTEST", "esewa_id": "9806800001",
           "password": "Nepal@123", "token": "123456",
           "success_url": "/gateway/verify", "failure_url": "/gateway/verify"}
```

```
ORDER = {
    "number": f"INV-{CUSTOMER['roll'].upper()}-05",
    "lines": [
        {"name": "Annual hosting plan", "qty": 1, "price": 3600},
        {"name": "Domain renewal", "qty": 1, "price": 1400},
    ],
    "delivery": 0,
}
```

```
payments = [] # every attempt: reference, amount, status, time
```

```
def order_total():
    return sum([l["qty"] * l["price"] for l in ORDER["lines"]] + ORDER["delivery"])
```

```
def new_reference():
    tail = "".join(random.choices(string.ascii_uppercase + string.digits, k=6))
```

```

return f"ESW-{CUSTOMER['roll'][-3:]}-{tail}"

@app.route("/")
def home():
    return render_template("checkout.html", customer=CUSTOMER, order=ORDER,
                           total=order_total(), sandbox=SANDBOX, payments=payments)

@app.route("/gateway", methods=["POST"])
def gateway():
    """Screen shown by the gateway. In production this page is on eSewa's site."""
    return render_template("gateway.html", customer=CUSTOMER, sandbox=SANDBOX,
                           amount=int(request.form.get("amount", 0)),
                           product_code=request.form.get("product_code", ""),
                           order_number=ORDER["number"])

@app.route("/gateway/verify", methods=["POST"])
def verify():
    """Check the credentials, then record the result of the payment."""
    amount = int(request.form.get("amount", 0))
    ok = (request.form.get("esewa_id") == SANDBOX["esewa_id"]
          and request.form.get("password") == SANDBOX["password"]
          and request.form.get("token") == SANDBOX["token"])

    payment = {
        "reference": new_reference(),
        "order": ORDER["number"],
        "payer": CUSTOMER["name"],
        "roll": CUSTOMER["roll"],
        "amount": amount,
        "channel": "eSewa ePay (sandbox)",
        "status": "COMPLETE" if ok else "FAILED",
        "time": datetime.now().strftime("%d %b %Y, %I:%M:%S %p"),
    }
    payments.append(payment)
    return redirect(url_for("result", reference=payment["reference"]))

@app.route("/result")
def result():
    reference = request.args.get("reference")
    payment = next((p for p in payments if p["reference"] == reference), None)
    return render_template("result.html", payment=payment, payments=payments,
                           customer=CUSTOMER)

if __name__ == "__main__":
    app.run(port=5005, debug=True)

```

lab5_payment/app.py

Output

The invoice for NPR 5,000 was paid through the sandbox gateway. It returned the reference ESW-003 with status COMPLETE, and the payment was written to the log. Being test mode, no live key is used and no money is transferred.

Lab 5 — payment gateway integration

The invoice amount is handed to eSewa running in sandbox (test) mode, and the gateway replies with a transaction reference and a status.

Invoice

INV-023BSKIT003-05 - billed to Abhishek Barali

DESCRIPTION	QTY	AMOUNT
Annual hosting plan	1	3600
Domain renewal	1	1400

Payable**NPR 5000****Pay NPR 5000 with eSewa****Sandbox configuration**

Values sent to the gateway with the payment request

Environment	TEST
Product code	EPAYTEST
Test eSewa ID	9806800001
Test password	Nepal@123
Test token	123456
Return URL	/gateway/verify

Sandbox only. No live merchant key is used, and no real money is transferred at any point of this flow.

Figure 10: Lab 5. The invoice page with the payable amount and the sandbox configuration sent to the gateway.

eSewa
TEST PAYMENT GATEWAY

AMOUNT TO PAY
NPR 5000
Merchant product code EPAYTEST
Invoice INV-023BSKIT003-05 - payer Abhishek Barali (023bscit003)

eSewa ID
9806800001

Password

Token (OTP)
123456

Confirm payment

Test credentials are pre-filled from eSewa's published sandbox account. Changing any of them makes the gateway return a FAILED status, which is how the failure path is demonstrated.

Sandbox environment - no live transaction

Figure 11: Lab 5. The gateway screen, which in production is hosted by eSewa, showing the amount and the pre-filled test credentials.

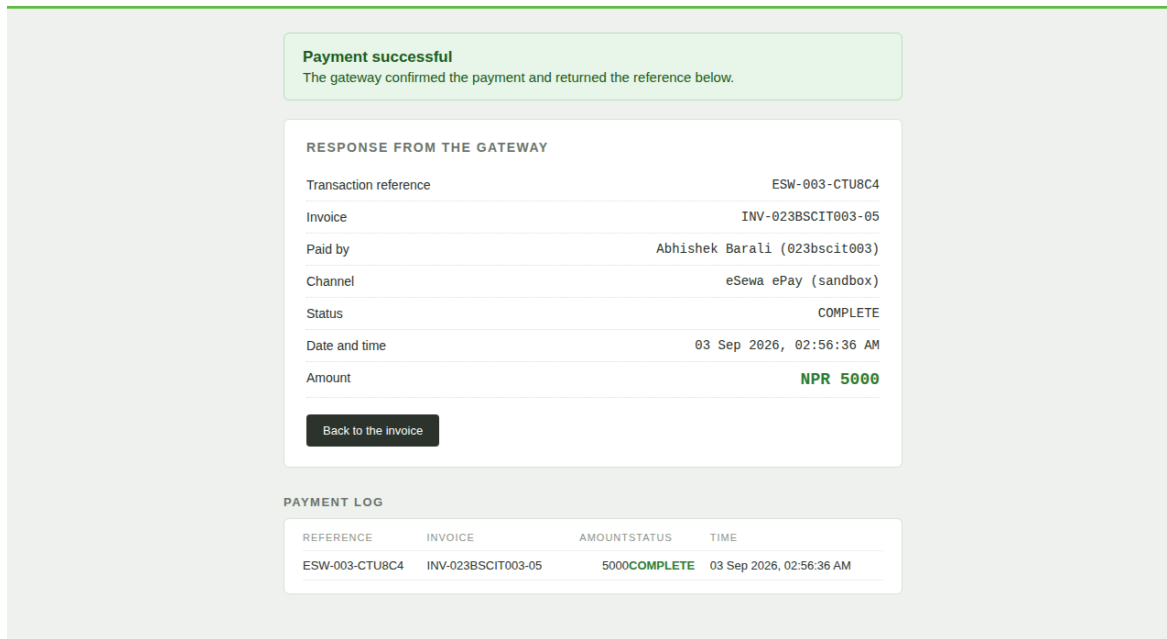


Figure 12: Lab 5 output. The gateway returned status *COMPLETE* with a transaction reference, and the payment log records the attempt.

Lab 6: Google Analytics Setup and Analytics Parameters

Studied on Google's official public GA4 demo, the Google Merchandise Store. No website was built and no tracking code was written for this lab; the demo is a fully working GA4 property that any Google user can add to their own Analytics, and it already carries real traffic and real e-commerce data from Google's own online store.

Objective

Sign in to Google Analytics, open the Merchandise Store demo, and study the main GA4 analytics parameters: users and sessions, engagement, events, conversions, and the e-commerce figures. Every screenshot below was taken from the live demo.

Property setup and Measurement ID

The demo property was added through Google's demo-account help page, which links the Google Merchandise Store GA4 property into the signed-in account with one click. Because it is a shared demo, the property already exists and is read-only, so nothing had to be created. Its Measurement ID, the G-XXXXXXXXXX handle a real site would put in its gtag.js snippet, is:

G-P2XN2NGD35

The Home screen confirms the property is live. Over the last 7 days it shows about 21K active users, 26K sessions and 20K key events, with a world map of where the users are (United States 5.6K, India 1.3K, Japan 599, Taiwan 327, Pakistan 185) and the top selling items with their revenue.

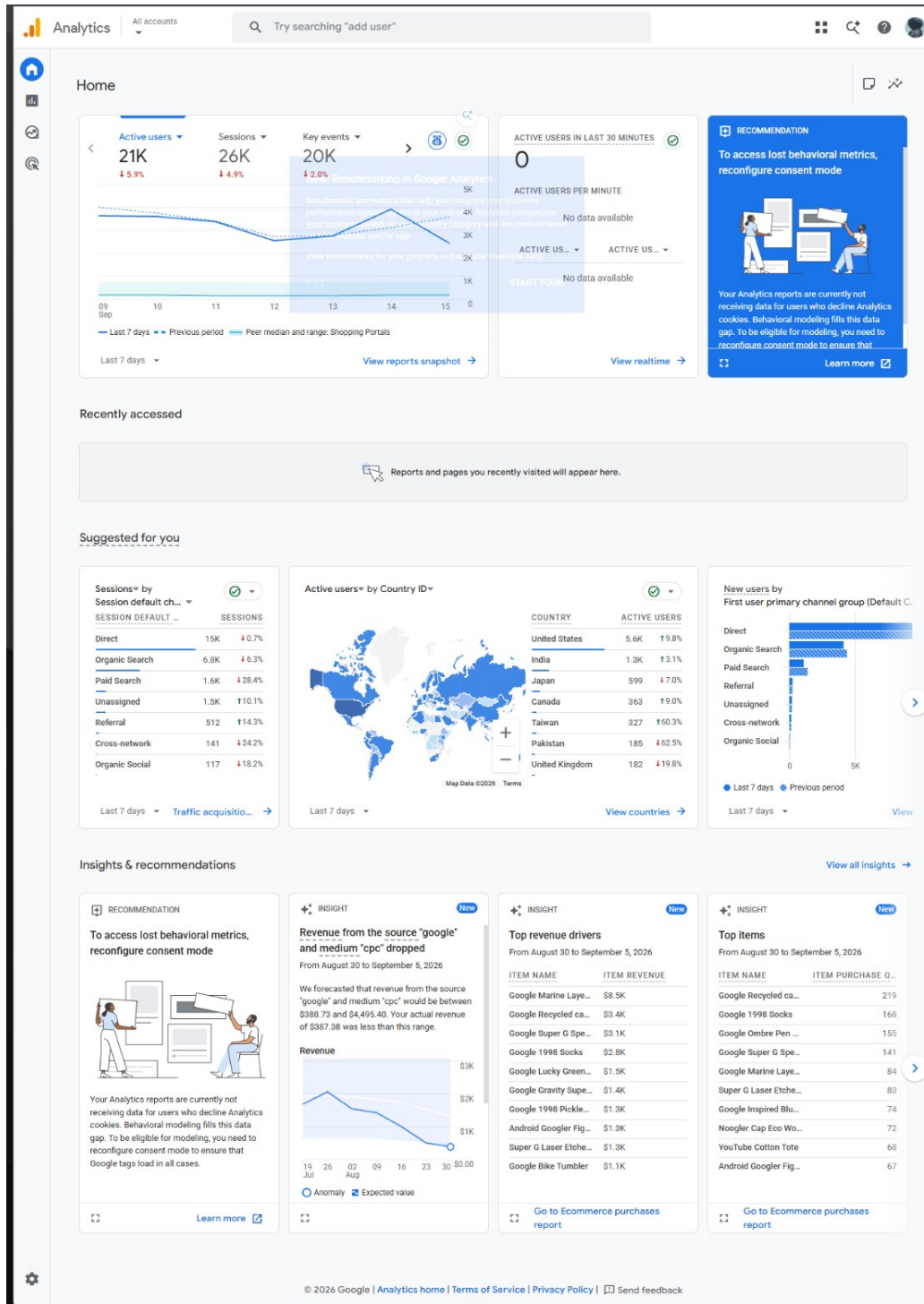


Figure 13: GA4 Home for the Google Merchandise Store demo: about 21K active users, 26K sessions and 20K key events over 7 days, with the live world map.

The demo data (Reports snapshot)

The Reports snapshot gathers the headline numbers for the last 28 days: about 88K active users, 80K new users, an average engagement time of 49 seconds, and 1.3M events. It also lists the top pages (Home at 101K views), the busiest cities (New York, Mountain View, Singapore, Lahore), the traffic sources, and audiences such as non-purchasers and likely 7-day purchasers. Home and the snapshot differ (21K versus 88K users) only because Home is showing 7 days and the snapshot 28 days; every GA4 report has its own date range.

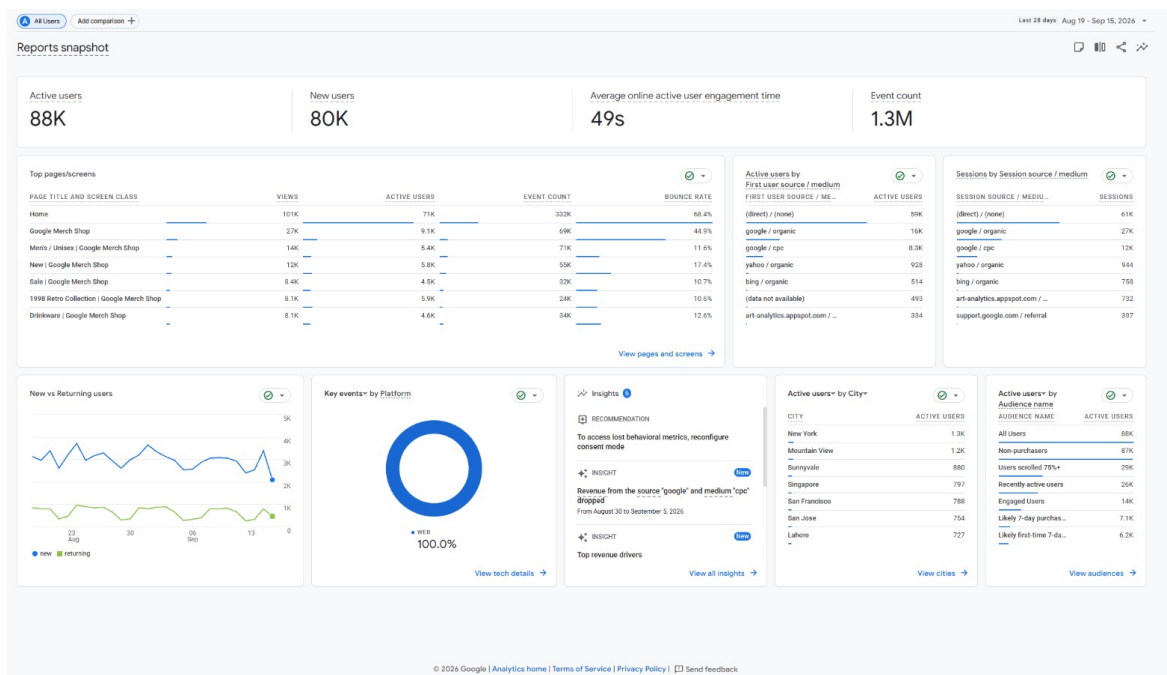


Figure 14: Reports snapshot: 88K active users, 80K new users, 49s average engagement time and 1.3M events, plus top pages, cities, sources and audiences (last 28 days).

Realtime report

The Realtime report shows the last 30 minutes as it happens. During the lab it showed about 67 active users, a world map of where they were, their source (mostly direct, some google), and the events firing live. The Realtime pages view broke the same traffic down by page: 69 active users and 243 views, with the home path busiest.

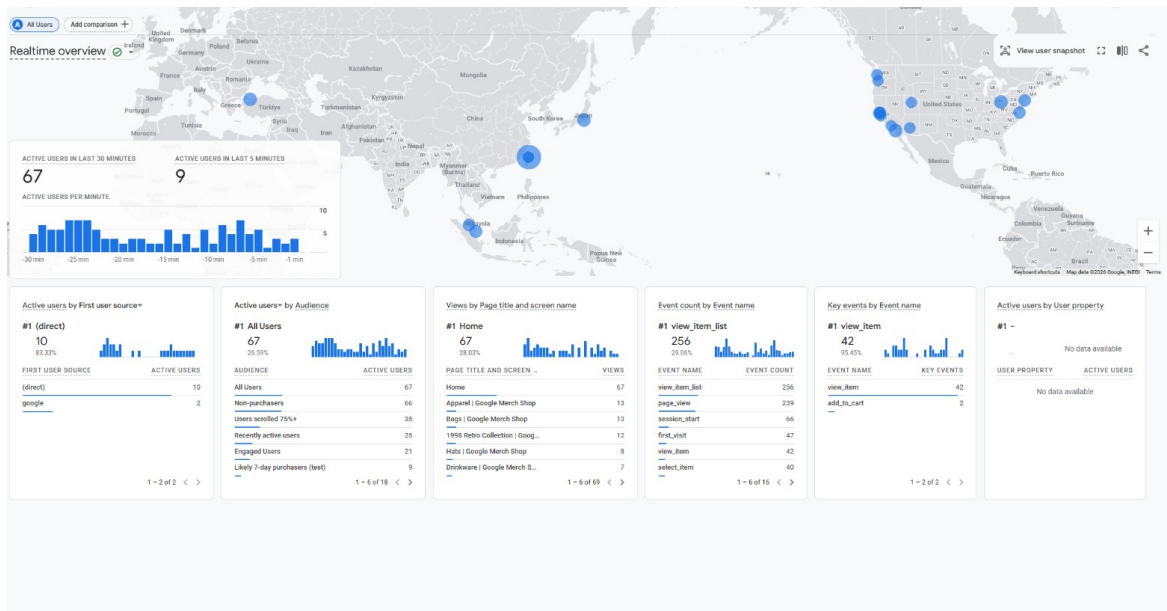


Figure 15: Realtime overview: about 67 active users in the last 30 minutes, their live locations, sources, and the events firing in real time.

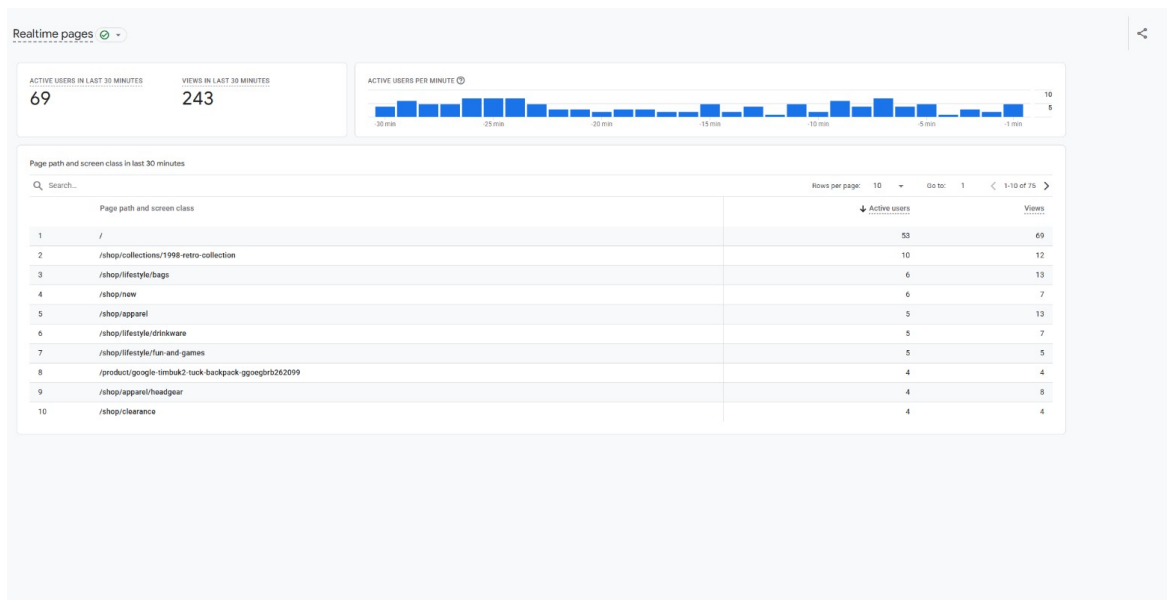


Figure 16: Realtime pages: 69 active users and 243 views in the last 30 minutes, broken down by page path.

Events report

GA4 is event-based: every interaction is an event. Over the 28 days there were about 1.26M events from 91K users, an average of 14.36 per user. The most common were page_view (344,109, about 27%), view_item_list (311,044, about 25%), session_start (108,642), first_visit (80,261) and view_item (74,245). Automatic events like session_start, first_visit, user_engagement and scroll appear without any extra setup.

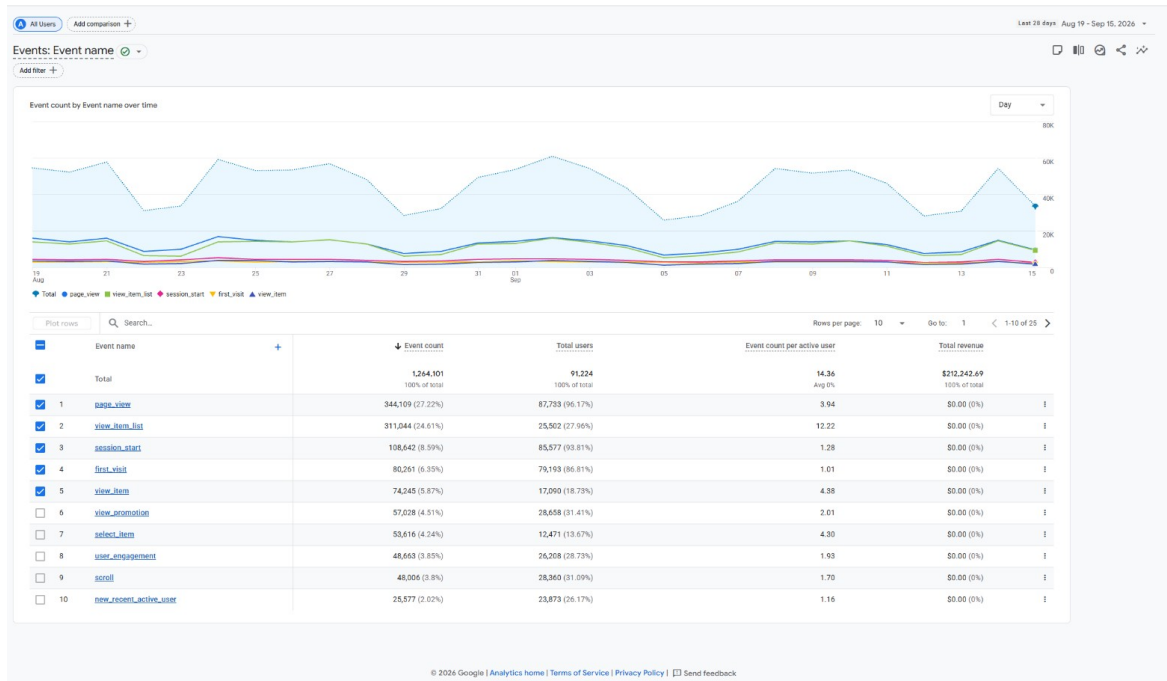


Figure 17: Events report: about 1.26M events over 28 days, led by page_view, view_item_list, session_start, first_visit and view_item.

Conversions (key events)

Conversions in GA4 are ordinary events marked as key events because they matter to the business, such as purchase, add_to_cart and view_item. The report showed about 85,871 key events and \$212,242.69 of event revenue. view_item from google/organic was the largest (31,574), and the purchase key event from direct traffic carried \$98,494.71 of revenue on its own.

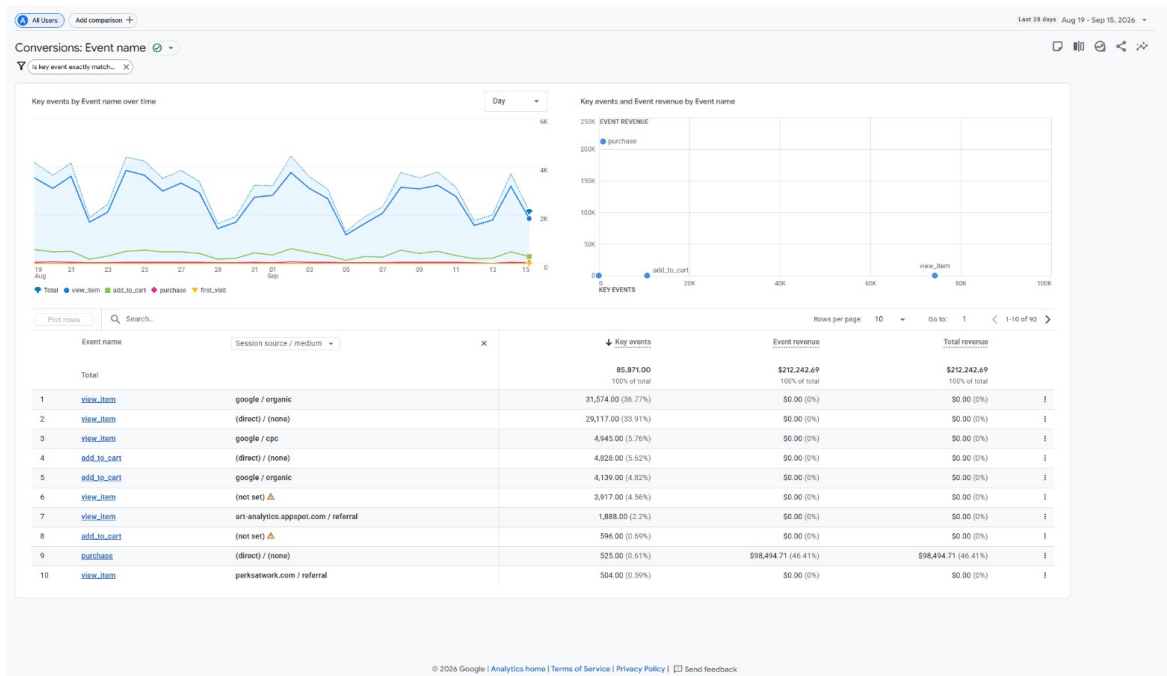


Figure 18: Conversions report: key events such as view_item, add_to_cart and purchase, with the purchase event carrying the revenue.

Monetization (e-commerce purchases)

The Ecommerce purchases report reads product performance. Across 28 days the store had 73,602 items viewed, 39,328 added to cart, 12,604 purchased, and \$185,866.84 in item revenue. Each row carries the product name, views, adds to cart, purchases and revenue. The best earner was the Google Marine Layer 1998 Pullover (3,049 views, 149 purchased, \$15,225.00), ahead of the Super G Gradient Tee and the Google 1998 Pickleball Set.

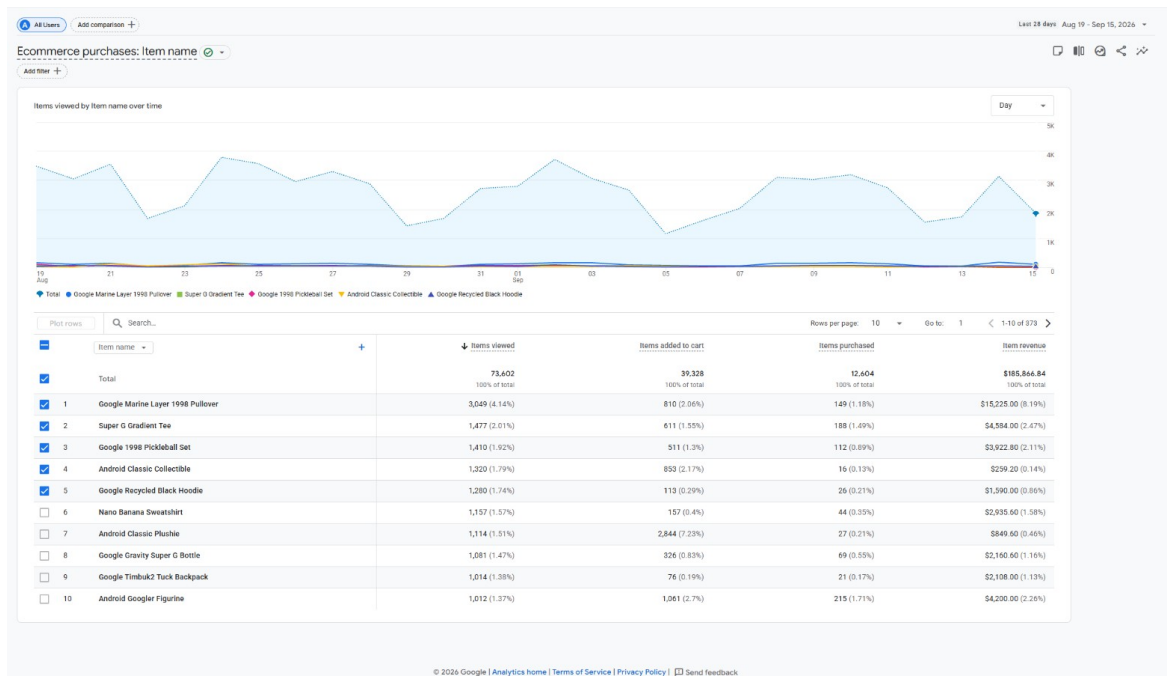


Figure 19: Ecommerce purchases: items viewed, added to cart, purchased and item revenue per product, led by the Google Marine Layer 1998 Pullover at \$15,225.

Analytics parameters explained

Users and Sessions: users are unique visitors (people); sessions are visits, and a session ends after 30 minutes of no activity. The demo showed about 88K active users and 80K new users over 28 days, so most traffic here was first-time visitors.

Engagement: engagement rate is the share of sessions that were meaningful (10 seconds or more, a key event, or 2+ views); average engagement time is how long the page was in the foreground. The demo's average was 49 seconds, with a large 'Users scrolled 75%+' audience, both signs of real engagement.

Events: everything is an event, from page_view, session_start, first_visit and user_engagement to e-commerce events like view_item, add_to_cart and purchase. Some fire automatically, some are set up by the site. The demo recorded about 1.26M events.

Conversions: a conversion is an event flagged as a key event because it matters, here purchase, add_to_cart and view_item. Marking a key event is what lets GA4 count conversions and attribute revenue; the purchase key event carried the store's revenue.

E-commerce parameters: these describe the products, item name, category, quantity, price and revenue. GA4 ties them to the item events, so one report shows how often a product was viewed, added to cart, bought, and how much it earned.

Observations from filtering

The reports were read across device, location and traffic source:

- By device: the 'Key events by Platform' card reads 100% Web, because the demo only has a web data stream (no app), so all activity is desktop and mobile browsers.
- By location: worldwide, but the United States dominates (5.6K active users), then India (1.3K), Japan (599), Canada (363), Taiwan (327) and Pakistan (185). Top cities were New York, Mountain View, Singapore and Lahore.
- By traffic source: direct traffic is largest (about 59K users, 61K sessions), then google organic search (16K users, 27K sessions), then google paid search (google/cpc, 8.3K).

Conclusion

The first five labs were built as separate working programs: Labs 1 to 4 cover the catalogue side of an online shop (categories and items, the cart, checkout into an order, and the wish list), each keeping its records in plain Python lists and dictionaries, and Lab 5 sends an invoice amount to a payment gateway running in sandbox mode and reads back the transaction reference and status. Lab 6 stepped outside coding to study Google Analytics 4 on Google's own Merchandise Store demo.

GA4 works on an event-based model: instead of counting only page views, it records every interaction as an event with its own parameters, and conversions are simply events marked as key events. Together the parameters describe behaviour and sales from both ends. Users, sessions and engagement show how many people come and how involved they are; events show what they do; conversions and the e-commerce figures show what it is worth. On the demo it was possible to see 88K users produce 1.26M events and \$185,866.84 in item revenue, and to trace which products, channels and places drove that result, all without owning a website or writing any tracking code.